EE6483 Artificial Intelligence and Data Mining
Mini Project (Option 1): Sentiments of Product Reviews

Group [TBD: group no.]

November 2026

Group members and contributions

Member	Matric no.	Contribution
A: [TBD: name]	[TBD:]	Shared code (`common.py`, data split, evaluation), TF-IDF baselines, noisy-label experiment, resource/accuracy analysis; report (a)(1), (b), (g), (i); report integration.
B: [TBD: name]	[TBD:]	GloVe features, averaged-GloVe + LR, BiLSTM model and tuning; report (a)(2), (a)(4), (d), (h).
C: [TBD: name]	[TBD:]	RoBERTa fine-tuning, ensemble and final submission, error analysis; report (a)(3), (c), (e), (f); README.

All members contributed papers to the literature table in (a)(2) and cross-reviewed each other's sections.

Contents

1 (a) Literature survey **2**
- 1.1 (a)(1) Problem definition, challenges and solution types . . . 2
- 1.2 (a)(2) Recent papers . . . 4
- 1.3 (a)(3) Top research groups . . . 4
- 1.4 (a)(4) Baseline choice and motivation . . . 4

2 (b) Feature format and data preprocessing **4**

3 (c) Model **6**

4 (d) Parameter selection **6**

5 (e) Test-set results **6**

6 (f) Analysis of correctly and incorrectly classified samples **6**

7 (g) Effect of the feature format on resources and accuracy **7**

8 (h) Hotel reviews without rating scores **9**

9 (i) Hotel reviews with noisy rating scores **9**

A Code and instructions **12**

1 (a) Literature survey

1.1 (a)(1) Problem definition, challenges and solution types

Problem definition. Sentiment analysis, also called opinion mining, is the computational study of the opinions, sentiments and emotions that people express in text [**?**]. A common formalisation describes an opinion as a quintuple (e, a, s, h, t): an entity e, one of its aspects a, the sentiment s towards that aspect, the opinion holder h and the time t. This project is the simplest instance of that definition: *document-level binary polarity classification.* The entity is the purchased product and the holder is the reviewer, both implicit, and each review receives one label for the whole document. Given a review x, we learn a classifier $f(x) \approx P(y{=}1 \mid x)$, where $y = 1$ is positive and $y = 0$ is negative, and predict $\hat{y} = \mathbb{1}[f(x) \geq \tau]$ for a decision threshold τ. Neutral or irrelevant reviews are not modelled.

Granularity. Sentiment can be analysed at three levels of granularity:

- *Document level*: one label per review, as in this project.
- *Sentence level*: usually a subjectivity filter followed by polarity classification.
- *Aspect level*: aspect-based sentiment analysis (ABSA) extracts aspects such as "fit" or "fabric" and assigns a polarity to each [**?**].

Related tasks use finer label sets, such as the five-point scale of the Stanford Sentiment Treebank, or emotion categories.

Challenges. The main difficulties, each illustrated with what we observe in our data:

1. **Negation and contrast.** Words such as "not", "never" and "but" change the polarity of the words around them. In "Not bad for just a shirt" (from our test set), "bad" signals a negative review, yet the review is mildly positive. A bag-of-words model loses the word order needed to resolve this.
2. **Implicit sentiment, sarcasm and irony.** Many reviews contain no explicit opinion word ("the seam came apart after one wash"), or say the opposite of what they mean.
3. **Mixed polarity.** A review may praise one aspect and criticise another. The document label then depends on which aspect the reviewer weighs more.
4. **Domain dependence.** The polarity of a word depends on the domain. "Small" is negative for a hotel room and often positive for a phone, and "unpredictable" is positive for a film plot but negative for a car's steering. Classifiers trained on one product category therefore degrade on another [**?**].
5. **Class imbalance.** Product reviews are mostly positive: 85.4% of our training reviews are positive and 14.6% negative. Predicting "positive" for every review already gives 85.4% accuracy, so we report Macro-F1 and the F1 of the negative class alongside accuracy.
6. **Noisy and informal text.** Reviews contain typos, slang, emoticons and very different lengths. Our training reviews have a median of 42 words, but the longest has 1,066. When star ratings are used as labels, some labels disagree with the text, which is the problem studied in (i).

Solution types under different settings. Table 1 organises common approaches along the three axes named in the assignment; Wankhade et al. [**?**] give a broader recent overview.

Supervised versus unsupervised. Supervised methods learn from labelled reviews. They range from Naive Bayes, maximum-entropy and SVM classifiers on bag-of-words features, and their strong NB-SVM variant, through convolutional and recurrent networks over word embeddings, to fine-tuning pre-trained Transformers such as BERT [**?**] and its successors. Unsupervised, lexicon-based methods need no labels:

Table 1: Solution types for sentiment classification under different settings. Our project is marked with ⋆.

Setting	Typical methods	Strengths / limitations
Supervised ⋆	BoW + NB/LR/SVM; CNN/RNN over embeddings; fine-tuned Transformers [**?**]	Most accurate when labelled in-domain data exist; needs labels and is domain-specific.
Unsupervised	Seed-word association; lexicons; rules (VADER)	No labels needed and interpretable; misses context, domain-specific terms and sarcasm.
Weak / semi-supervised	Distant labels from emoticons or ratings; self-training; LLM prompting [**?**]	Cheap labels at scale; labels are noisy (see (i)).
Closed-set ⋆	Softmax over a fixed label set	Simple; inputs from unseen classes are forced into a known class.
Open-set	One-vs-rest rejection (DOC); confidence thresholds	Can reject neutral or irrelevant inputs; needs extra thresholds.
No domain shift ⋆	Standard train/validation/test protocol	Validation accuracy predicts test accuracy.
Domain shift	SCL pivots [**?**]; denoising autoencoders; DANN; domain-adaptive pre-training	Uses unlabelled target text; gains depend on how far apart the domains are.

- scoring phrases by their association with seed words such as "excellent" and "poor";
- looking words up in a sentiment lexicon such as SentiWordNet;
- rule-based scoring with VADER.

They transfer easily across domains but are usually less accurate than supervised models. Between the two extremes lie weakly supervised methods. Distant supervision uses emoticons or star ratings as noisy labels, and semi-supervised methods use self-training on unlabelled reviews. Zero-shot and few-shot prompting of large language models is competitive on simple binary polarity, but still falls behind fine-tuned models on fine-grained and structured sentiment tasks [**?**].

Closed-set versus open-set. In the closed-set setting every test review belongs to one of the training classes, here positive or negative. In the open-set setting, test inputs may come from classes never seen in training: neutral reviews, spam, text that is not a review, or another language. A closed-set classifier still forces each of these into "positive" or "negative". Open-set methods add a rejection option. DOC replaces the softmax with one-vs-rest sigmoids and rejects an input when no class is confident enough. Confidence-based detection flags inputs whose maximum softmax probability is low.

Without versus with domain shift. When training and test data come from the same distribution, standard supervised learning applies. Under domain shift, labelled data from a source domain (e.g. books) must serve an unlabelled target domain (e.g. kitchen appliances). Approaches include:

- structural correspondence learning, which aligns domain-specific words through shared "pivot" features [**?**];
- stacked denoising autoencoders that learn domain-invariant representations;
- domain-adversarial training (DANN);
- continued pre-training of the language model on target-domain text.

These methods are the basis of our answer to (h).

Our setting. The project is *supervised, closed-set and in-domain.* Training and test reviews come from the same product-review distribution; 101 of the 1,851 test reviews even appear verbatim in

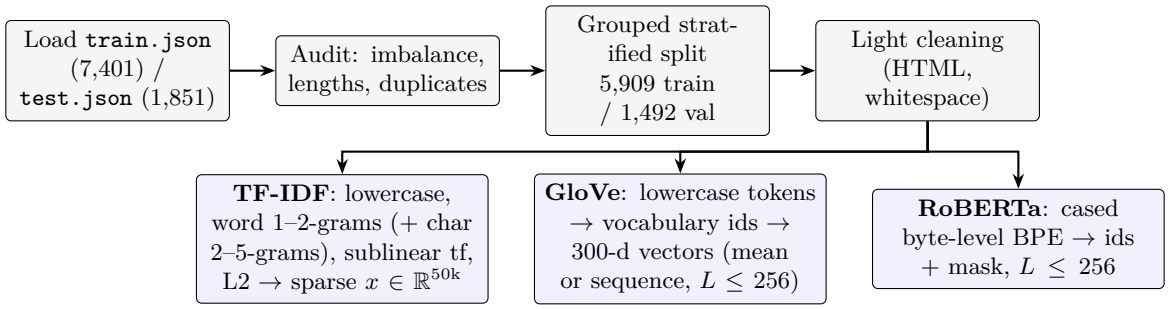


Figure 1: Data preprocessing pipeline. Every vectoriser, vocabulary and model is fitted on the training part only; the validation split is used only for model selection and the test set only for the final predictions.

the training set. The data are imbalanced, and the labels may contain some noise. We therefore build supervised classifiers. A TF-IDF linear model serves as the baseline, and sequence models and pre-trained language models serve as improvements; (a)(4) motivates this choice. We use threshold tuning and Macro-F1 to handle the imbalance.

1.2 (a)(2) Recent papers

[TBD: member B]

1.3 (a)(3) Top research groups

[TBD: member C]

1.4 (a)(4) Baseline choice and motivation

[TBD: member B]

2 (b) Feature format and data preprocessing

Choice of feature format. We compare three feature formats of increasing cost, one per model line, so that (g) can measure the trade-off directly:

1. **Sparse bag-of-n-grams with TF-IDF weighting**, used by linear classifiers;
2. **static pre-trained word embeddings** (300-dimensional GloVe [**?**]), used by an averaged-embedding classifier and by a BiLSTM;
3. **subword tokens fed to a pre-trained Transformer** (RoBERTa-base), whose contextual representation is fine-tuned end-to-end. This is the format of our final model.

Figure 1 summarises the pipeline. All three formats share the loading, auditing, splitting and cleaning steps described below.

1. Loading and auditing. Both files are read with `pandas.read_json`. The training set has 7,401 reviews with a binary label (6,319 positive, 1,082 negative). The test set has 1,851 unlabelled reviews. No review is missing or empty. Review length varies widely: the median is 42 words, 75% of reviews have at most 70 words, 97% at most 178, and the longest has 1,066 words. We also checked for duplicates, comparing texts after lower-casing and collapsing whitespace:

- 154 distinct texts occur more than once in the training set, covering 373 rows, and every copy of a text has the same label;
- 101 test reviews are identical to a training review;
- 21 test reviews duplicate another test review.

2. Grouped, stratified validation split. Every model is tuned on one shared validation split. If duplicates were split at random, copies of the same review would land on both sides, and the validation score would partly measure memorisation. We therefore group rows by their normalised text and take fold 0 of a 5-fold `StratifiedGroupKFold` (seed 42). This yields 5,909 training and 1,492 validation reviews. The validation set contains 218 negatives (14.6%, the same ratio as the full set), and no text appears on both sides. Cross-validation inside the training part, including the internal folds used to calibrate the SVM, uses the same grouping. The test set is used only to produce `submission.csv`.

3. Light text cleaning. We unescape HTML entities, collapse runs of whitespace and strip leading and trailing spaces. We deliberately do *not*:

- remove stop words, because standard stop lists contain "not", "no" and "but", which carry sentiment;
- stem, or strip punctuation such as "!" and "?".

Text is lower-cased for TF-IDF and GloVe, and kept in its original case for RoBERTa, whose tokenizer is case-sensitive.

4a. TF-IDF features (L1). The text is tokenised with a pattern that keeps single-character tokens (e.g. the "t" of "don't") as well as "!" and "?". We extract unigrams and bigrams; bigrams capture short negations such as "not good" and "not worth". Each term t in review d is weighted with sublinear term frequency and smoothed inverse document frequency,

$$w_{t,d} = (1 + \log \mathrm{tf}_{t,d}) \cdot \left(\log \frac{1+N}{1+\mathrm{df}_t} + 1\right), \tag{1}$$

where N is the number of training reviews and df_t the number of reviews containing t. Each review vector is then L2-normalised. The vectoriser is fitted on the training rows only. Three settings were tuned by grouped 5-fold cross-validation:

- the minimum document frequency (1, 2 or 3);
- the vocabulary cap (10k, 30k, 50k or unlimited);
- whether to append character 2–5-grams, which can absorb misspellings.

The selected representation has 50,000 word 1–2-gram features. It is very sparse: 86 non-zeros per review on average, a density of 0.17%.

4b. Static word embeddings (L2). [TBD: B: please check and adjust this paragraph.] Reviews are lower-cased and tokenised into words. The vocabulary is built on the training part (minimum frequency 2) plus `<pad>` and `<unk>` tokens. Each token is mapped to its 300-dimensional `glove.6B` vector [**?**], pre-trained on 6 billion tokens of Wikipedia and Gigaword. Words without a GloVe vector are [TBD: randomly initialised / mapped to `<unk>`]. Their share of validation tokens is [TBD: OOV rate]. For the averaged-embedding classifier, a review is the mean of its token vectors, a dense $\mathbb{R}^{300}$ input. For the BiLSTM, a review is a sequence of token ids truncated to 256 tokens and padded only to the longest review in each mini-batch.

4c. Subword tokens for a pre-trained Transformer (L3). [TBD: C: please check and adjust this paragraph.] Reviews keep their original casing and are tokenised with RoBERTa's byte-level BPE tokenizer, which has a vocabulary of 50,265 subwords. Every string can be encoded, so there are no out-of-vocabulary words. The sequence is wrapped in `<s>` … `</s>` and truncated to 256 subword tokens, keeping the beginning of the review; this affects only about 3% of reviews. Mini-batches are padded dynamically, and an attention mask marks the real tokens. The model learns the features itself: each token receives a 768-dimensional contextual vector, and the `<s>` vector feeds the classifier (see (c)).

5. Labels and class imbalance. Labels are used as given (0 = negative, 1 = positive). Each model outputs $P(y{=}1 \mid x)$. We report metrics at threshold 0.5 and at thresholds tuned on the validation split: one maximising accuracy (`thr_acc`, used for the submission) and one maximising Macro-F1 (`thr_f1`). Class-weighted losses were tried as a hyper-parameter in every model line, as described in (d).

3 (c) Model

3.1 TF-IDF classifiers

The three TF-IDF classifiers take the sparse n-gram vectors described in part (b). Logistic regression maps a weighted sum of the features to a positive-class probability. The linear SVM produces a margin score, which is converted to a probability by sigmoid calibration on grouped folds within the training split. Complement Naive Bayes estimates each class's feature weights from reviews in the other class, with additive smoothing.

The feature and classifier settings were selected by grouped five-fold cross-validation within the training portion. Each selected model was then fitted on 5,909 reviews and evaluated on the shared validation set of 1,492 reviews. For test prediction, both the TF-IDF vectoriser and the classifier were refitted on all 7,401 labelled reviews.

3.2 GloVe embeddings and BiLSTM

The mean-embedding classifier averages the GloVe vectors available for the tokens in a review. If none of the tokens has a pre-trained vector, the review is represented by a zero vector. The resulting 300 features feed a regularised logistic regression classifier.

The BiLSTM uses the same training vocabulary to initialise an embedding matrix of size $V \times 300$, where V is the vocabulary size. Entries without pre-trained vectors are initialised from a zero-mean Gaussian with standard deviation 0.05, and the padding row is fixed at zero. The input is a batch of integer token IDs of shape $B \times L$, where B is the batch size and $L \leq 256$.

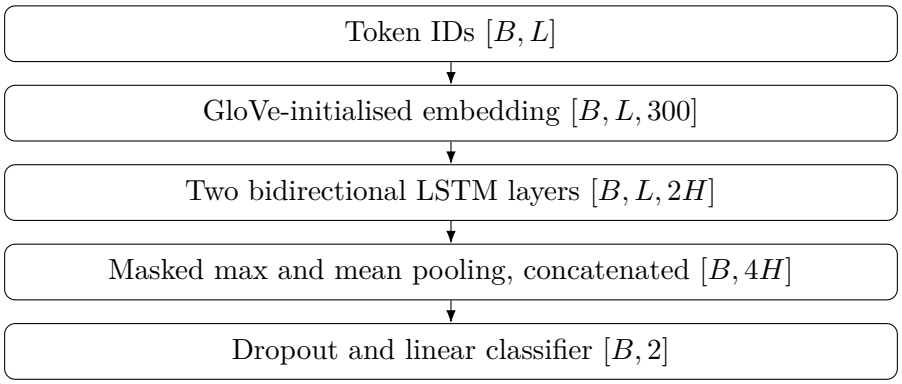


Figure 2: GloVe-initialised BiLSTM. H denotes the hidden width per direction.

In each layer, the backward direction reverses only the valid part of the sequence. Both pooling operations ignore padding. Max pooling keeps the strongest activation in each channel, while mean pooling summarises the valid states. The two pooled vectors are concatenated to give the classifier $4H$ features, or 512 when $H = 128$.

The BiLSTM was trained with Adam and cross-entropy. In the weighted variant, class j receives a loss weight of $n/(2n_j)$, where n is the number of training reviews and n_j is the number in that class. The best epoch was selected by validation Macro-F1 at a threshold of 0.5. Softmax gives the positive-class probability, and the validation and test probabilities were averaged across seeds 42, 43, and 44.

3.3 RoBERTa-base

Input and classifier. After the light cleaning described in part (b), each review retains its original casing. The RoBERTa tokenizer splits the text into subwords and limits the sequence to 256 tokens, including the special tokens. Longer sequences are truncated at the end. Within each batch, shorter sequences are padded to match the longest one, and an attention mask distinguishes the input tokens from padding.

The encoder produces a 768-dimensional representation at each token position. The classification head reads the representation at the first token, `<s>`. A dense layer with `tanh` activation transforms this vector, with dropout before and after the layer during training. The final linear layer gives one score per class. Softmax converts the two scores to probabilities, and the positive-class probability is retained for prediction.

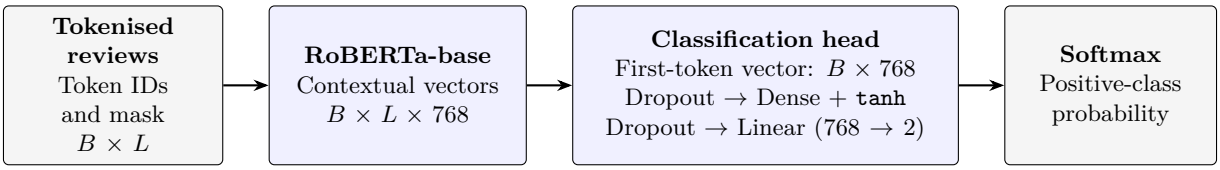


Figure 3: RoBERTa-base architecture for review classification. B is the batch size and $L \leq 256$ is the sequence length after padding within the batch. The head uses the encoder output at the first token, `<s>`.

Fine-tuning. The encoder and classification head were fine-tuned together on the 5,909 training reviews from the shared split. Each run lasted one epoch in FP32 with a batch size of 4. AdamW used a fixed learning rate of 2×10^{-5} and a weight decay of 0.01. The loss was unweighted cross-entropy.

Prediction. The data split and training settings were fixed across the three runs, with random seeds 42, 43, and 44. For each review, the three positive-class probabilities were averaged before applying the decision threshold. Part (e) explains how the threshold was chosen on the validation set and reports the resulting test predictions.

4 (d) Parameter selection

[TBD: member B]

5 (e) Test-set results

Final model. We averaged the positive-class probabilities from three RoBERTa-base runs with random seeds 42, 43, and 44. A weighted combination of RoBERTa (75%) and TF-IDF + SVM (25%) raised validation macro-F1 from 90.82% to 91.63%. The gain of 0.81 percentage points was smaller than the standard deviation of 20.27 percentage points across the three individual RoBERTa runs at a threshold of 0.500. Under our model selection rule, we used the averaged RoBERTa probabilities for the final submission.

Decision threshold. Among the thresholds evaluated on the 1,492 labeled validation reviews, 0.680 gave the highest accuracy (95.38%) and macro-F1 (90.82%) for the averaged RoBERTa probabilities. We applied the same threshold to the test set. Reviews with an average positive-class probability of at least 0.680 received label 1 (positive). The remaining reviews received label 0 (negative).

Test predictions. The table shows the predicted label distribution for the 1,851 unlabeled test reviews.

Predicted label	Number of reviews	Share
Positive (1)	1,578	85.25%
Negative (0)	273	14.75%
Total	1,851	100.00%

The file `submission.csv` contains the `reviews` and `predictions` columns in the original test-set order.

6 (f) Analysis of correctly and incorrectly classified samples

Correct and incorrect predictions. RoBERTa matched the final human labels on 31 of the 40 sampled reviews (77.5%). The sample deliberately included reviews near the decision threshold and reviews on which the models disagreed, so this accuracy applies only to the selected reviews.

RoBERTa correctly classified reviews 274 and 99. In review 274, the reviewer points out a limitation of the garment and still recommends it for smoothing under clothes. That recommendation supports the positive label. In review 99, the reviewer initially liked the bra's shape and support, but the underwire eventually broke through. This failure supports the negative label despite the initial praise.

RoBERTa classified review 549 as positive even though the reviewer reported a broken boot strap and asked about returning the boots. It classified review 1123 as negative. In that review, the cracking and breaking described by the reviewer concerned the old plastic luggage tags. The reviewer was satisfied with the new leather tag, which was the product being evaluated.

7 (g) Effect of the feature format on resources and accuracy

The feature format decides two things: what information reaches the classifier (word identity only, word similarity, or word meaning in context), and how much computation and memory are needed to obtain it. We first compare the three formats in general terms (Table 2), then report what we measured on this dataset (Table 3), and finally isolate the effect of feature dimensionality inside the TF-IDF line (Table 4).

Dimensionality within one format. Table 4 shows clearly diminishing returns. Growing the TF-IDF vocabulary from 1k to 30k n-grams raises Macro-F1 from 0.811 to 0.850 and AUC from 0.930 to 0.951. Beyond 30k the gain stops: keeping all 123k n-grams changes AUC by less than 0.001. Because the representation is sparse, the cost grows slowly with the nominal dimension. Each review activates only 45–98 entries, and the training matrix stays under 7 MB. Appending character 2–5-grams is the exception. It multiplies the non-zeros per review by about 7 (649 vs. 98), the matrix size by 6.5, and the training time by 5. The validation Macro-F1 rises by about 0.01, but grouped 5-fold cross-validation shows no gain (0.8514 with character n-grams vs. 0.8515 without). A difference of 0.01 Macro-F1 is within the noise of a split with only 218 negatives, so the cheaper word-only representation was selected.

Across formats: accuracy. The three formats differ in how much linguistic structure they expose. TF-IDF already reaches 92.8–93.0% accuracy, because most reviews contain strong, explicit opinion words. The largest LR weights are intuitive: "disappointed", "uncomfortable", "returned" and "poor" for negative reviews, "great", "comfortable", "perfect" and "love" for positive ones. However, the unigram "not" alone receives the most negative weight. As a result, "not bad" gets only a small positive bigram weight (+0.59), which barely offsets the strongly negative "not" and "bad", while "not worth" (-3.31) is captured correctly. This is exactly where order-aware and contextual

Table 2: Comparison of the three feature formats used in this project.

	TF-IDF n-grams (L1)	Static GloVe embeddings (L2)	Contextual subwords, RoBERTa (L3)
Representation	Sparse, one dimension per n-gram (here 50k; 0.17% non-zero)	Dense, 300 per token; mean-pooled or sequential	Dense, 768 per subword token, context-dependent
Word order / context	Only local, via bigrams	Mean: none. BiLSTM: learned sequentially	Full self-attention over 256 tokens
Similar words	Unrelated dimensions ("great" vs "excellent")	Close in vector space	Close, and disambiguated by context
Out-of-vocabulary words	Dropped (2.1% of validation tokens, 24% of word types)	`<unk>` or random vector ([TBD: B: OOV rate])	None: unknown words are split into subwords
Trainable parameters	≈ dimension + 1 (50k)	Mean+LR: 301. BiLSTM: ≈0.8M + embeddings	≈125M
Hardware / training cost	CPU, seconds	CPU, minutes (BiLSTM)	GPU, minutes to hours
Memory	Sparse matrix of a few MB	Embedding table $V \times 300$ (the full GloVe file is ≈1 GB)	≈0.5 GB of weights plus activations
Interpretability	High: one weight per n-gram	Low	Low (needs attribution methods)
Data needed	Moderate; benefits from more data	Less, thanks to pre-trained vectors	Least for a given accuracy; knowledge transferred from pre-training

features should help: negation, contrast ("but the…" is among the top negative features) and implicit sentiment. [TBD: After B and C push results: state the measured gain of BiLSTM and RoBERTa over TF-IDF and whether it exceeds the seed standard deviation; cross-reference the error categories in (f).]

Across formats: resources. Resource use grows much faster than accuracy. TF-IDF + LR trains in about 1.5 s on a CPU with about 120 MB peak memory, and predicts the 1,492 validation reviews in a few milliseconds. The GloVe line needs the pre-trained embedding file. The full `glove.6B.300d` table has 400k×300 floats, and only the rows of our vocabulary are kept. The BiLSTM adds sequential computation that is [TBD: B: measured slowdown] slower than TF-IDF. RoBERTa-base has about 125M parameters, roughly 2,500 times the TF-IDF+LR model. It needs a GPU for practical fine-tuning ([TBD: C: time, peak GPU memory]), and inference cost grows with the sequence length because of attention. Figure 2 plots accuracy against training time on a log scale.

Practical implications.

- **Small labelled data favours pre-trained features.** With only 5.9k training reviews, TF-IDF must learn every n-gram's polarity from scratch, and 24% of the word types in the validation reviews never occur in training. Pre-trained embeddings, and above all pre-trained language models, bring knowledge learned from billions of words, so they generalise better to rare words, typos and paraphrases.
- **Large data or tight budgets favour sparse features.** TF-IDF scales linearly, runs on any CPU and is fully interpretable. It is the right choice when latency, cost or explainability matter more than the last percentage point, and it is a strong baseline.

Table 3: Measured accuracy and resource use on the shared validation split (1,492 reviews, 218 negative). Training time excludes loading data and pre-trained weights. TF-IDF runs are deterministic, so their seed standard deviation is 0. Neural models are averaged over seeds 42/43/44. Hardware: [TBD: all: confirm] TF-IDF on an AMD laptop CPU (16 threads).

Model	Feature dim	Params	Acc@0.5	Acc@τ_{acc}	Macro-F1@τ_{f1}	Train (s)	Peak mem (MB)
TF-IDF + LR	50,000	50,001	0.924	0.928	0.851	1.5	122
TF-IDF + LinearSVC	50,000	150,009[†]	0.929	0.930	0.857	2.8	150
TF-IDF + ComplementNB	10,000	20,000	0.910	0.926	0.857	1.2	155
Mean GloVe + LR	300	301	[TBD: B]	[TBD: B]	[TBD: B]	[TBD: B]	[TBD: B]
GloVe + BiLSTM	300/token	[TBD: B]	[TBD: B]	[TBD: B]	[TBD: B]	[TBD: B]	[TBD: B]
RoBERTa-base	768/token	≈125M	[TBD: C]	[TBD: C]	[TBD: C]	[TBD: C]	[TBD: C]

[†]Three calibrated SVMs, one per internal calibration fold, each with its own weight vector.

Table 4: Effect of TF-IDF dimensionality, with the classifier fixed (LR, C=10, balanced class weights), trained on the training part and evaluated on the validation split.

Features	Dim	Non-zeros / review	Matrix (MB)	Vectorise + fit (s)	Macro-F1@0.5	AUC
Word 1–2-grams, top 1k	1,000	45	3.1	1.2	0.811	0.930
Word 1–2-grams, top 5k	5,000	63	4.3	1.2	0.840	0.946
Word 1–2-grams, top 10k	10,000	70	4.8	1.1	0.845	0.949
Word 1–2-grams, top 30k	30,000	81	5.5	1.3	0.850	0.951
Word 1–2-grams, top 50k	50,000	86	5.8	1.3	0.845	0.950
Word 1–2-grams, all	123,515	98	6.7	1.2	0.846	0.951
+ char 2–5-grams (all)	174,735	649	43.9	6.3	0.858	0.952

- **Robustness to domain change differs.** TF-IDF and GloVe fix their vocabulary at training time. A subword model covers any input string, which matters for the hotel-review scenario in (h).

In short, for this task the feature format moves accuracy by a few points but cost by orders of magnitude. The choice should therefore follow the accuracy target and the available hardware.

8 (h) Hotel reviews without rating scores

[TBD: member B]

9 (i) Hotel reviews with noisy rating scores

The question refers to "the same sentiment classification problem in (g)". We read this as the hotel-review project of (h): the reviews now come with rating scores, but some of those scores are wrong. Training labels derived from the ratings are therefore *noisy*.

How our classifier would be affected

Sources and types of noise. Rating errors on review sites are rarely uniform [**?**]. Three kinds can be distinguished:

- *Random errors*, such as mis-clicks or swapped scales, are roughly *symmetric*.
- *Systematic errors* are *class-conditional*. A guest may write a positive text but give a low score because of the price or a single incident, or a hotel may receive inflated scores. Noise then runs mainly in one direction.
- *Instance-dependent errors* concentrate on ambiguous, mixed or sarcastic reviews, which are exactly the hard cases.

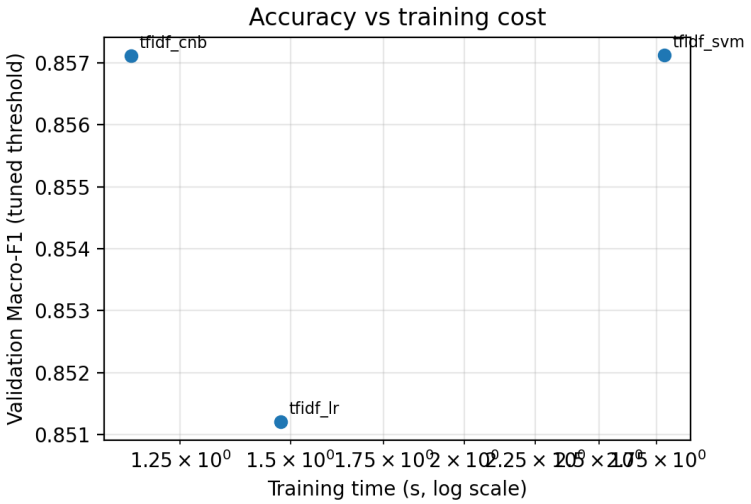


Figure 4: Validation Macro-F1 (tuned threshold) versus training time (log scale). Regenerated by `src/resource_table.py` from every model's `metrics.json`. [TBD: final version after all outputs are in]

Effect on training. A discriminative classifier fits the labels it is given. Over-parameterised neural networks can memorise even completely random labels. They do tend to learn the clean, dominant patterns first and only later fit the noise. So a fine-tuned RoBERTa or a BiLSTM trained to convergence on noisy ratings would gradually absorb the errors, and its decision boundary and confidence estimates would suffer. Early stopping on clean data delays this but does not prevent it. A regularised linear TF-IDF model cannot memorise as freely, but its weights are biased in the same way. Class imbalance amplifies the damage. With an 85:15 class ratio, flipping each label independently with probability 20% means only 41% of the reviews labelled "negative" are truly negative, because flipped positives outnumber the real negatives. The minority class, usually the one that matters most to a hotel, is affected first.

Effect on evaluation. If the validation and test labels are noisy too, measured accuracy underestimates the true accuracy, and model selection and threshold tuning chase the noise. A small, manually verified clean set is therefore essential for any of the remedies below.

Experiment: injecting label noise into our data

To quantify these effects we corrupted the labels of our 5,909 training-part reviews and kept the 1,492-review validation split clean. We used two noise models:

- *symmetric*: every label is flipped with probability 0, 5, 10 or 20%;
- *positive→negative*: only positive labels are flipped, with probability 5, 10 or 20%, mimicking a positive text given a low score.

The base model is TF-IDF (word 1–2-grams) + LR, and each setting is repeated with 3 seeds. Methods M1–M3 rely on 5-fold out-of-fold probabilities $p(x)$ computed on the noisy training part, with the same grouped folds as in (b). We compared four strategies:

- **M0, baseline**: train on the noisy labels.
- **M1, sample selection (Confident Learning)** [**?**]. For each class j, set the threshold t_j to the mean predicted probability of j over samples labelled j. A sample is flagged when its most probable class among those with $p_k \geq t_k$ differs from its given label. Flagged samples are removed and the model is retrained.

Table 5: Validation results under injected label noise (mean ± std over 3 seeds). Left block: Macro-F1 at threshold 0.5. Right block: ROC-AUC, which does not depend on the threshold. "Neg. prec." is the fraction of training reviews labelled negative that are truly negative. Without class weights and threshold tuning, the clean baseline's Macro-F1 (0.789) is lower than the tuned TF-IDF model in (g) (0.851).

		Neg.	Macro-F1 @ 0.5				ROC-AUC			
Noise	Rate	prec.	M0	M1	M2	M3	M0	M1	M2	M3
–	0%	1.00	.789	**.810**	.765	.692	**.950**	.948	.946	.945
Symmetric	5%	0.77	.789	**.814**	.789	.698	.938	**.944**	**.944**	**.944**
	10%	0.60	.772	**.805**	.765	.690	.910	.936	.929	**.939**
	20%	0.41	.720	**.792**	.754	.724	.837	**.909**	.878	.907
Pos→neg	5%	0.77	.802	**.823**	.812	.728	.935	**.944**	**.944**	**.944**
	10%	0.63	.806	**.819**	.809	.769	.917	.940	.936	**.941**
	20%	0.46	.778	**.819**	.796	.814	.901	.931	.917	**.933**

Standard deviations are 0.003–0.020 for Macro-F1 and below 0.008 for AUC; full values are in `outputs/noise_exp/summary.csv`.

- **M2, label correction**: a label is flipped when the out-of-fold model assigns more than 0.9 probability to the opposite class.
- **M3, re-weighting**: each sample is weighted by the out-of-fold probability of its given label.

Findings. Table 5 and Figure 3 lead to five observations.

1. **Noise hurts, and imbalance makes it worse.** Under 20% symmetric noise the baseline's AUC drops from 0.950 to 0.837, and its Macro-F1 from 0.789 to 0.720. The F1 of the negative class falls from 0.63 to 0.52.
2. **Threshold-0.5 metrics can mislead.** Positive→negative noise of 5–10% slightly *raises* the baseline's Macro-F1 at 0.5, although its AUC falls. The extra negative labels merely shift the decision boundary towards the minority class. This is why we also report AUC, and why thresholds must be tuned on clean data.
3. **Sample selection (M1) is the most reliable method.** It gives the best Macro-F1 in every setting, and at 20% symmetric noise it recovers AUC from 0.837 to 0.909. It removed 1,633 reviews on average; 44% of them had actually been flipped, and it caught 61% of all flipped labels. Even on the clean data it removed about 80 hard or ambiguous reviews and improved Macro-F1 by 0.02.
4. **Label correction (M2) is precise but conservative.** 83–97% of the labels it changed were truly wrong, but it changed only 200–280 of them, so its gains are moderate.
5. **Re-weighting (M3) ranks well but needs re-calibration.** It matches M1 in AUC and is best under strong positive→negative noise. However, its Macro-F1 at 0.5 is poor at low noise: negative reviews receive lower out-of-fold confidence, so re-weighting shrinks the minority class further. Combined with class weights or threshold tuning, this weakness would disappear.

These results come from synthetic noise, a linear model and a single validation split. They show the size of the problem and the relative behaviour of the strategy families, not the gains to expect on real hotel ratings.

Three approaches to improve the algorithm

The three families below follow the taxonomy of the recent survey by Song et al. [**?**], where the individual methods named here are reviewed in detail.

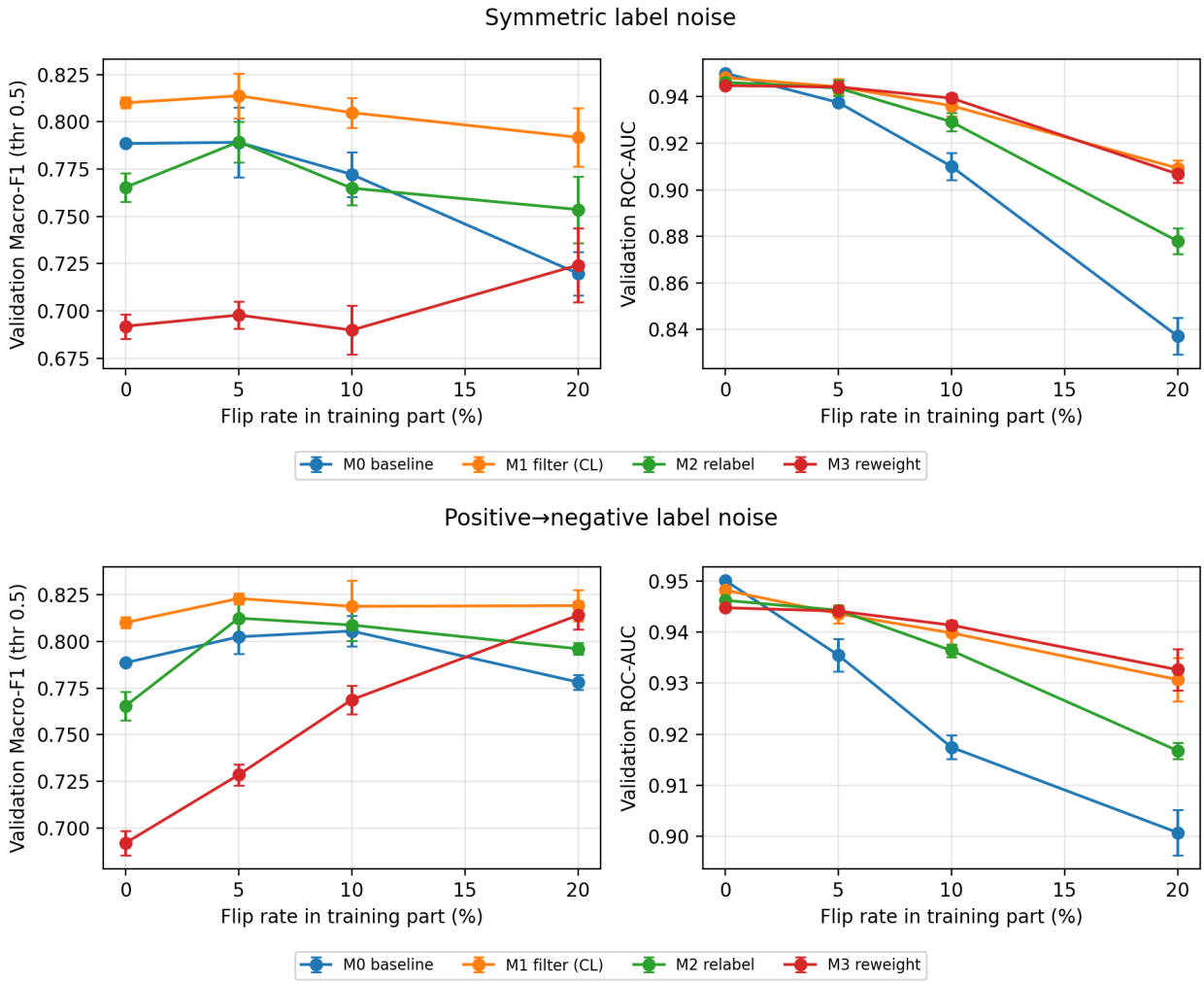


Figure 5: Validation Macro-F1 at threshold 0.5 (left) and ROC-AUC (right) as the training-label flip rate grows. Error bars show ±1 std over 3 seeds.

1. **Noise-robust losses and loss correction.** Replace plain cross-entropy with a loss that penalises confidently wrong labels less. Options include generalized cross-entropy, which interpolates between cross-entropy and the noise-tolerant MAE; symmetric cross-entropy; and label smoothing. When the noise is class-conditional, as with positive texts given low scores, one can instead estimate the 2×2 noise-transition matrix T and correct the loss through it (forward/backward correction). T can be estimated from a small clean subset. These methods change only the loss function of our RoBERTa fine-tuning, and add no extra cost.

2. **Sample selection and data cleaning.** Exploit the fact that networks fit clean samples first, so small-loss samples are more likely to be correctly labelled. Co-teaching trains two networks and lets each one update on the small-loss samples chosen by the other. Confident Learning estimates the joint distribution of noisy and true labels from out-of-fold probabilities, then prunes or ranks the suspicious samples [**?**]; this is our M1, the best method in our experiment. Flagged reviews can also be sent to annotators instead of being discarded.

3. **Label correction and semi-supervised learning.** Instead of discarding suspicious samples, correct or down-weight them. Our M2 and M3 are simple versions. DivideMix fits a two-component mixture model to the per-sample losses to separate likely-clean from likely-noisy reviews. It then treats the noisy ones as *unlabelled* data and trains on both parts with MixMatch-style semi-supervised learning. For hotel reviews, the text itself is a second, independent signal. A zero-shot model, for example our RoBERTa classifier or an LLM, can check

each rating against its text, and the two can be combined into a corrected label.

Recommended pipeline for the hotel project.

1. Manually verify a few hundred reviews to obtain a clean development set, and use it to estimate the noise rate and the transition matrix T.
2. Fine-tune a pre-trained Transformer with a robust loss (GCE or label smoothing) and early stopping on the clean set.
3. Run Confident Learning on out-of-fold predictions; relabel or drop the flagged reviews, sending the uncertain ones to annotators.
4. Retrain, optionally with DivideMix-style semi-supervised learning on the doubtful part, and tune the decision threshold on the clean set.

A Code and instructions

The code is in the `src/` folder of the submitted archive; `README.md` lists the commands to reproduce every result. [TBD: C: final README summary]